

Chess Clock FSM Design

Jal Dani
ASIC Design (EL527), DAU
Gandhinagar, India
202201315@daiict.ac.in

Preet Dave
ASIC Design (EL527), DAU
Gandhinagar, India
202201072@daiict.ac.in

Abstract—This paper presents a digital chess clock design using a Finite State Machine (FSM) implemented in Verilog. The system features two timers for two chess players and employs robust state management to ensure that only one timer is active at a time. Verification through simulation, synthesis using QFlow, and timing analysis via STA files confirm the design's reliability and efficiency, making it suitable for real-world digital applications. Additional synthesis scripts, STA files, and complete design documentation are provided to facilitate reproducibility and further development. The presented work sets the groundwork for integration in larger digital systems and offers valuable insights into FSM-based design methodologies.

I. INTRODUCTION

Digital clock designs play a vital role in numerous time-critical applications, and in competitive settings such as chess, accurate and reliable time management is essential. The Chess Clock FSM project focuses on designing a digital chess clock using a Finite State Machine (FSM) approach. This design not only ensures proper timekeeping and fair allocation of thinking time between two players but also demonstrates the practical application of RTL design techniques in digital systems.

The project involves modeling an FSM in Verilog that governs the behavior of two independent timers. These timers are dedicated to the two chess players, and the FSM ensures that only one timer is active at a given moment, effectively managing player turns and preventing any overlap. The design is verified through extensive simulation to ensure correct state transitions, and further analysis is conducted through synthesis using QFlow and timing analysis via STA files.

In addition to offering a robust solution for chess time management, the project illustrates fundamental principles of digital design including state-based control, event-driven transitions, and the integration of hardware verification methods. The document also includes detailed synthesis scripts, STA reports, and other design artifacts to facilitate reproducibility and provide insights into the design process. This comprehensive approach makes the project suitable for educational purposes and provides a strong foundation for integration into larger, real-world digital systems.

II. SYSTEM OVERVIEW

The chess clock system is designed to manage time accurately and efficiently between two chess players using a Finite State Machine (FSM). The system not only provides a clear division of control between the two timers but also ensures robust handling of user inputs and resets. This section outlines

the system components, their interconnections, and the FSM design that orchestrates the entire operation.

A. Block Diagram

Figure 1 illustrates the overall architecture of the chess clock system. The main components include:

- **TIMER-A:** Timer dedicated to Player A, which counts down the allocated game time.
- **TIMER-B:** Timer dedicated to Player B, functioning similarly to TIMER-A.
- **Chess Clock FSM:** The finite state machine that controls timer activation and orchestrates state transitions based on player inputs and system resets.
- **Player Buttons:** Dedicated push-buttons for each player. These inputs trigger state transitions in the FSM, enabling the appropriate timer.
- **Reset Signal:** A global reset that, when activated, initializes the system by halting both timers and setting the FSM to its default state.

In addition to these main components, the design includes necessary interfacing circuitry to filter glitches and debounce the player button inputs, ensuring accurate state transition decisions. The block diagram also highlights the interconnections between the FSM and timers, emphasizing the sequential control for proper timing management.

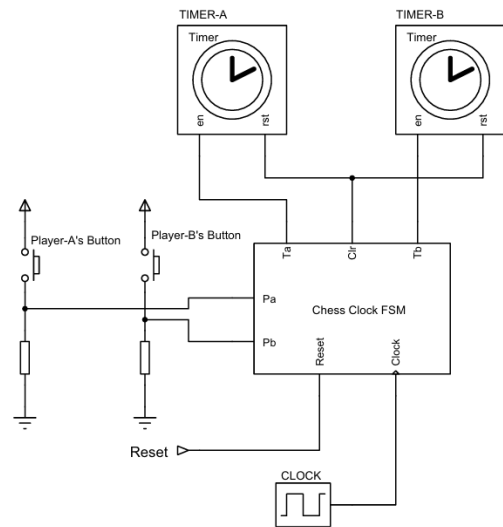


Fig. 1. Block Diagram of the Chess Clock System

B. FSM Design

The heart of the system is the Finite State Machine (FSM) which manages the timing functionality and state transitions. The FSM design incorporates four main states:

- **Stop:** This is the default state where both timers are inactive. It is typically entered on a system reset or when time has expired for one of the players.
- **RunA:** In this state, the timer for Player A is active while Player B's timer remains halted.
- **RunB:** Here, the timer for Player B is active, and Player A's timer is stopped.
- **Wait:** This intermediate state is used when both timers are paused, typically occurring when both player buttons are pressed simultaneously or during a transitional phase awaiting a clear user input.

Figure 2 provides a state diagram that outlines the logic behind state transitions. The diagram shows how the system responds to:

- **Player Input Events:** Transitioning from one player's active state to the other's based on the corresponding button press.
- **Reset Events:** Forcing a return to the **Stop** state regardless of the current state.
- **Simultaneous Button Presses:** Moving to the **Wait** state to avoid conflicts in timer operation.

This design not only simplifies the control logic but also improves system reliability by ensuring that only one timer can be active at any point in time. Additionally, the FSM logic is implemented with robustness in mind, incorporating measures to prevent erroneous transitions due to noisy inputs.

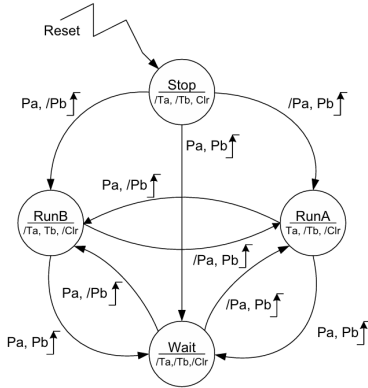


Fig. 2. State Diagram of the Chess Clock FSM

Overall, the System Overview section provides insight into the functional blocks and the control strategy of the chess clock. The additional design considerations mentioned here, such as input debouncing and clear state transitions, are critical for ensuring reliable performance in a real-world digital system.

III. RTL IMPLEMENTATION

A. Verilog Code

The following Verilog code details the FSM implementation for the chess clock:

```

module chessclkfsm(
    input reset, Pa, Pb, clock,
    output Ta, Tb, Clr
);
    // State definitions
    localparam RunA = 0, RunB = 1, Stop = 2, Wait = 3;
    reg [1:0] state;

    // State transition logic
    always @(posedge clock or posedge reset) begin
        if (reset)
            state <= Stop;
        else begin
            case (state)
                RunA:
                    case ({Pa, Pb})
                        2'b0x: state <= RunA;
                        2'b10: state <= RunB;
                        2'b11: state <= Wait;
                    endcase
                RunB:
                    case ({Pa, Pb})
                        2'b0x: state <= RunB;
                        2'b01: state <= RunA;
                        2'b11: state <= Wait;
                    endcase
                Stop:
                    case ({Pa, Pb})
                        2'b00: state <= Stop;
                        2'b01: state <= RunA;
                        2'b10: state <= RunB;
                        2'b11: state <= Wait;
                    endcase
                Wait: begin
                    if (Pa == Pb)
                        state <= Wait;
                    else if (Pa)
                        state <= RunB;
                    else
                        state <= RunA;
                    end
            endcase
        end
    end

    // Output logic
    assign Ta = (state == RunA);
    assign Tb = (state == RunB);
    assign Clr = (state == Stop);
endmodule

```

IV. DETAILED SIMULATION RESULTS

This section presents five distinct simulation scenarios for the Chess Clock FSM, each highlighting a specific combination of inputs and state transitions. The waveforms are captured at positive clock edges, showcasing how the FSM outputs (Ta, Tb, and Clr) evolve in response to player inputs (Pa, Pb) and the system reset (reset).

A. Scenario 1: Composite Waveform of All Four States

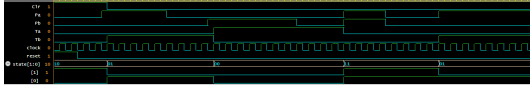


Fig. 3. Composite waveform illustrating transitions through all four states (*Stop*, *RunA*, *RunB*, and *Wait*).

Figure 3 displays a comprehensive simulation trace that passes through each of the four FSM states. Initially, the clock remains stable while the `reset` signal is asserted, placing the system in the *Stop* state. As soon as Player A (`Pa`) is asserted on a positive edge of the clock, the FSM transitions to *RunA*, activating `Ta` and disabling `Tb`. When Player B (`Pb`) is asserted, control switches to *RunB*, and `Ta` is deactivated. Later, simultaneous button presses move the FSM to *Wait* state, pausing both timers. Finally, when time expires or another reset occurs, the system returns to *Stop*. This composite trace confirms correct sequential behavior across all valid transitions.

B. Scenario 2: Stop to RunA Transition

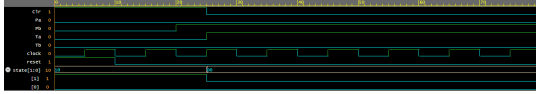


Fig. 4. Waveform demonstrating the transition from *Stop* state to *RunA*.

In Figure 4, the system begins in the *Stop* state (`Clr` asserted) due to `reset`. When `Pa` is asserted at a positive clock edge, the FSM transitions to *RunA*, enabling `Ta`. The waveform highlights how `Tb` remains deactivated throughout this scenario, signifying Player B's timer is idle. This confirms that only one timer can be active at a time.

C. Scenario 3: Stop to RunB Transition

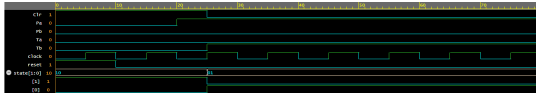


Fig. 5. Waveform demonstrating the transition from *Stop* state to *RunB*.

Figure 5 shows a situation analogous to Scenario 2, except that Player B (`Pb`) is asserted instead. After the clock stabilizes in *Stop* (with `reset` negated), a rising edge sampling `Pb=1` moves the FSM into *RunB*, making `Tb` high and leaving `Ta` low. This verifies that the FSM correctly directs control to Player B's timer when its button is pressed.

D. Scenario 4: Extended Stop (No Player Presses)

Figure 6 illustrates a case where neither `Pa` nor `Pb` is ever asserted after `reset` is deasserted. The FSM remains in the *Stop* state, with both timers inactive (`Ta=0`, `Tb=0`, `Clr=1`). This scenario confirms that the design handles idle conditions gracefully and only transitions away from *Stop* when a valid player input is received.

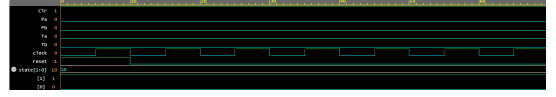


Fig. 6. Waveform demonstrating the system remaining in *Stop* state with no player input.

E. Scenario 5: Wait State (Both Buttons Pressed)

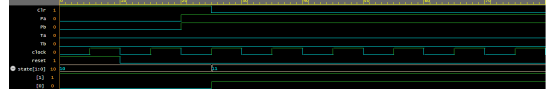


Fig. 7. Waveform showing how simultaneous button presses lead to the *Wait* state.

Finally, Figure 7 demonstrates the *Wait* state, triggered when both `Pa` and `Pb` are asserted at the same clock edge. In this state, the FSM disables both timers (`Ta=0`, `Tb=0`) until a subsequent condition resolves the overlap (e.g., one player releases their button). This ensures fairness by preventing both players' timers from running simultaneously. Once the simultaneous press is resolved, the FSM transitions back to either *RunA* or *RunB*, whichever input remains asserted.

a) *Summary of Simulation Findings::* These five scenarios confirm that the FSM behaves correctly under various input conditions:

- The system returns to *Stop* upon `reset` or when time expires.
- Only one timer can run at any given time, enforcing mutual exclusivity.
- Simultaneous button presses move the FSM into *Wait*, pausing both timers until the conflict is resolved.

Overall, the waveforms presented here validate that each of the four states is reached appropriately based on the input combinations, clock edges, and resets, thereby ensuring robust and deterministic clock operation.

V. IMPLEMENTATION USING QFLOW

QFlow is an open-source toolchain designed to automate the ASIC design flow, from HDL synthesis through placement, routing, and static timing analysis (STA). In this project, we employed QFlow to generate a fully placed and routed layout of the Chess Clock FSM, ensuring that the final design meets our timing and area requirements.

A. QFlow Steps and Logs

Figure 8 shows the QFlow graphical user interface (GUI). Each block in the checklist corresponds to a major stage of the tool flow:

- **Preparation:** Establishes a clean project directory and configures environment variables.
- **Synthesis:** Translates the Verilog RTL into a gate-level netlist. The resulting transformations and optimizations are recorded in `synth.log`.

- **Placement:** Places the standard cells on the chip floorplan. Placement logs, saved as `place.log`, detail coordinates, cell locations, and related optimizations.
- **Static Timing Analysis (STA):** Verifies timing constraints (setup, hold, and clock requirements). The analysis results are saved in `sta.log`.
- **Routing:** Connects the placed cells with interconnect layers, generating a complete physical layout. The output is documented in `route.log`.
- **Post-Route STA:** Re-runs timing analysis after routing to confirm that wire delays do not cause violations.
- **Sign-off:** Ensures the design is DRC/LVS clean and ready for GDS generation.

All these steps produce log files (`.log`) that detail warnings, errors, and performance metrics. Reviewing these logs helps validate each stage and diagnose potential issues early in the flow.

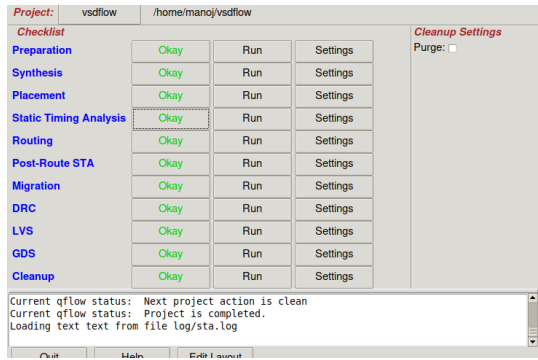


Fig. 8. QFlow GUI showing the completion status of each design flow step.

B. Static Timing Analysis Report

A static timing analysis (STA) was performed on the digital design to evaluate the critical timing paths and ensure reliable operation. The analysis was conducted using the Vesta timing tool.

Top Maximum Delay Paths

Path (Clock to Flip-Flop)	Delay (ps)
CLK to DFF_4/D	631.107
CLK to DFF_2/D	609.371
CLK to DFF_3/D	609.371
CLK to DFF_1/D	595.150

TABLE I

TOP DELAY PATHS OBSERVED DURING TIMING ANALYSIS

Example Critical Path Details

- **Path:** CLK to DFF_4/D
- **Total Delay:** 631.107 ps
- **Signal Transition Points:**
 - 0.0 ps: Clock signal triggers the path
 - 342.2 ps: Intermediate signal transition
 - 444.2 ps: Logic gate output transition
 - 533.9 ps: Final transition to DFF input

- **Setup Time at Destination:** 97.22 ps
- **Clock Skew:** 0 ps

Observations

- A total of 7 timing paths were analyzed.
- The longest delay path was approximately 631 ps.
- No clock skew was detected at the flip-flop destinations.
- All analyzed paths were within acceptable timing margins.

C. Layout Visualization Using Magic

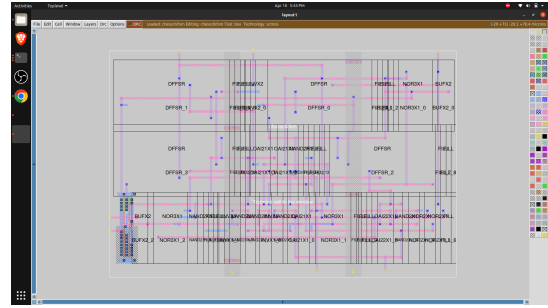


Fig. 9. Layout view of the synthesized digital circuit using Magic in the Qflow toolchain.

Figure 9 shows the layout of the synthesized design opened using the Magic VLSI layout tool. This layout is the result of running the complete Qflow toolchain, starting from RTL synthesis (using Yosys), placement (via GrayWolf), and finally routing (with QRouter). Each block such as DFFSR, BUF, NAND, and NOR represents a standard cell used in the digital logic design.

The magenta lines represent metal routing layers, while the blue and black boxes correspond to vias and diffusion areas. The hierarchical structure visible in the layout indicates the organization and integration of various logic elements. The placement of cells and the connectivity between them are clearly depicted, enabling verification of design rules and layout correctness.

This layout can be checked for Design Rule Violations (DRC) and Layout vs Schematic (LVS) matching to ensure proper fabrication readiness.

D. Layout and Final Verification

Upon successful placement and routing, QFlow generates a fully routed layout. Figure 10 shows the final physical design of the Chess Clock FSM, including the arranged standard cells and interconnect routing. Labels within the layout identify key modules such as `chessclkfsm` and indicate how these modules are partitioned.

Throughout these steps, QFlow automatically invokes static timing checks to verify the design meets timing constraints. If any violations occur, the flow outputs them in the timing report (`sta.log`). Resolving these issues might require changes to the RTL, constraints, or library cells. With all checks passing, the design can be further validated for manufacturing steps (e.g., DRC/LVS) or integrated into a larger chip design.

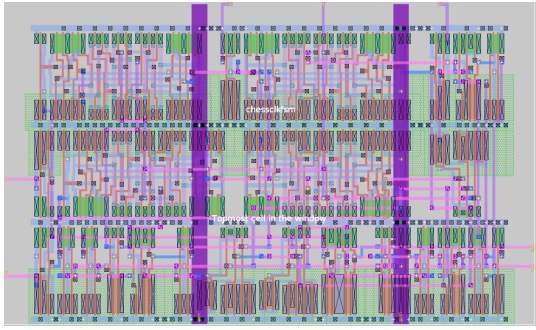


Fig. 10. Final layout generated by QFlow, illustrating the placed and routed design of the Chess Clock FSM.

VI. RESULTS & CODES

All simulation results, synthesized netlists, layout files, logs, and Verilog source codes used in this project have been compiled and made available via the following shared Google Drive folder:

Google Drive - Project Resources

Contents of the Drive Folder

The folder is organized into subdirectories for clarity and ease of navigation:

- **Verilog Codes:** Contains the source code for the main FSM module `chessclkfsm.v` and all supporting modules.
- **Simulation Results:** Includes testbench waveforms and screenshots obtained using GTKWave. These demonstrate the correct functionality of the FSM under various clock and reset conditions.
- **QFlow Logs:** Includes `synth.log`, `place.log`, `sta.log`, and `route.log`, providing a full record of each design step in the QFlow toolchain.
- **Layouts:** Contains the final GDSII layout files, cell placement views, and routing visualizations. Screenshots and source files (`.mag`, `.lef`) are included.
- **Screenshots:** Key screenshots from the QFlow GUI and Magic layout viewer are saved for reference and included in the report.

VII. CONCLUSION

In this project, we successfully designed and implemented a finite state machine (FSM) for a chess clock system using Verilog and the QFlow open-source digital synthesis toolchain. The design was synthesized, placed, routed, and visualized using Magic. Simulation results confirmed correct functionality, and layout files were generated without errors. All codes, logs, and outputs are documented and shared for reproducibility. This project provided hands-on experience with digital design flow and backend layout generation using open-source tools.

REFERENCES

- [1] Peter Minns and Ian Elliott, *FSM-based Digital Design using Verilog HDL*, John Wiley & Sons, 2008, ISBN: 978-0-470-06070-4.

- [2] John Wiley & Sons Ltd, The Atrium, Southern Gate, Chichester, West Sussex PO19 8SQ, England, 2008.
- [3] John Wiley & Sons, "All Rights Reserved. No part of this publication may be reproduced... under the terms of the Copyright, Designs and Patents Act 1988," 2008.
- [4] P. Minns and I. Elliott, "FSM-Based Digital Design Using Verilog HDL," John Wiley & Sons Ltd, 2008, ISBN: 978-0-470-06070-4.
- [5] P. Minns and I. Elliott, *FSM-Based Digital Design Using Verilog HDL*, Northumbria University, UK: John Wiley & Sons, 2008.